

B4ゼミ 補足資料

本課題のアーキテクチャ

Vercelでデプロイされた静的なWebサイトにユーザがアクセスし、ユーザが登録した情報がSupabaseを通じてデータベースに格納される。

今回扱うアーキテクチャを参考に、ゲーム課題におけるシステム設計を行っていただきたい。

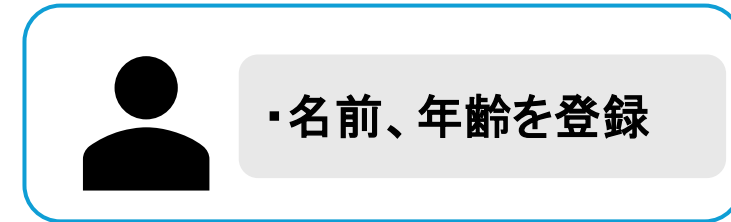
1.ローカル環境



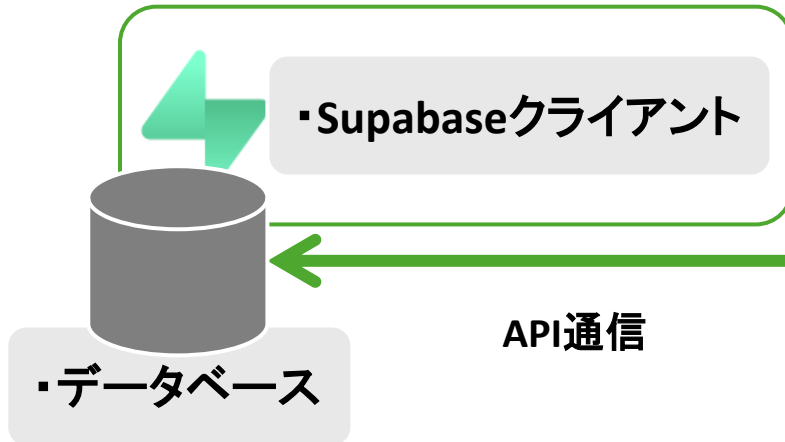
2.GitHub



5.ユーザ



4.Supabase



3.Vercel



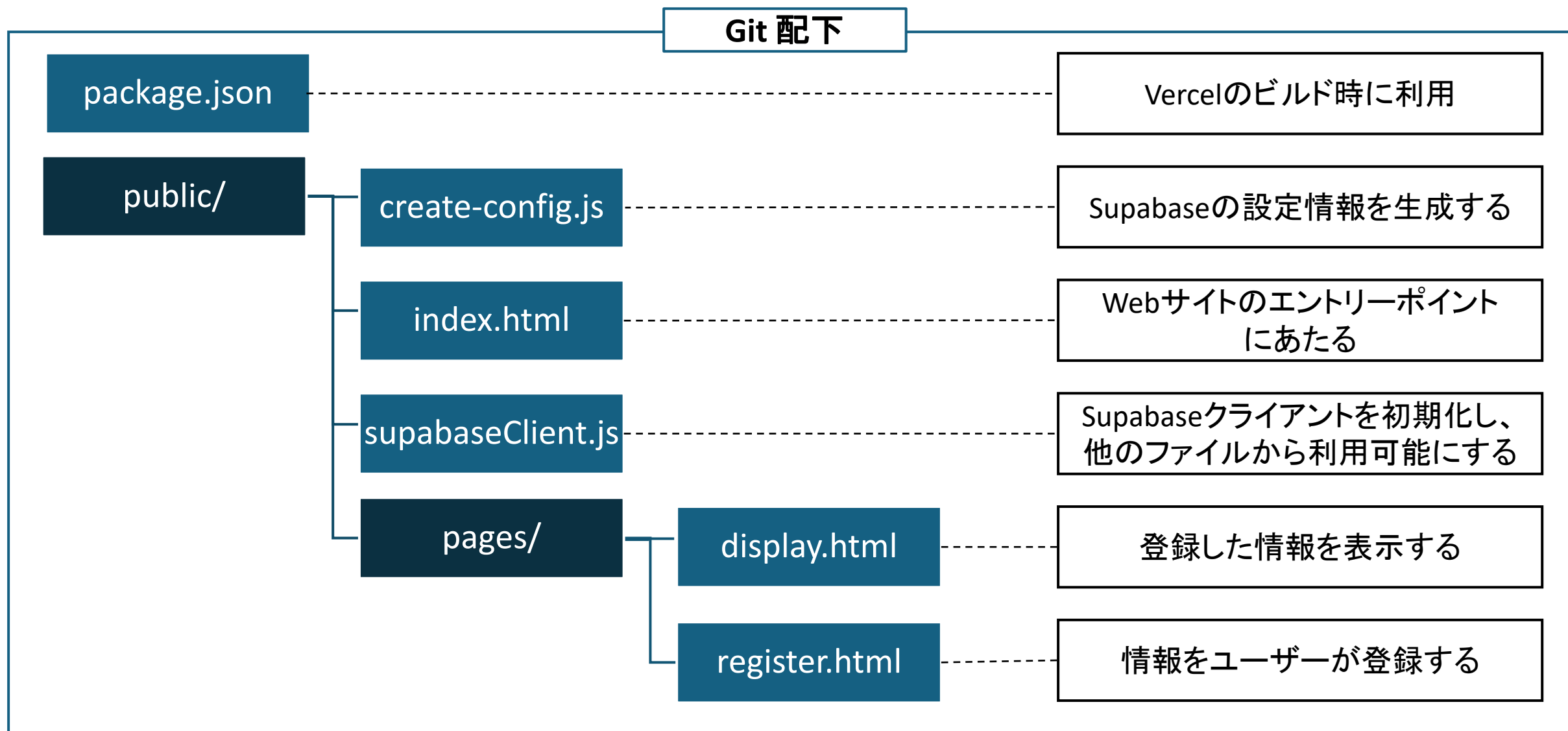
git push

自動デプロイ

API通信

アクセス

各ファイルの構成、役割について



データベースの基本操作について

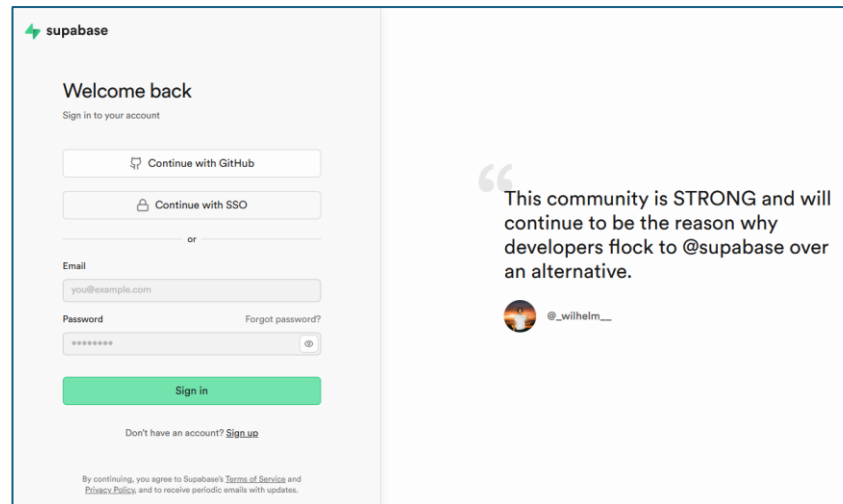
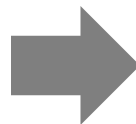
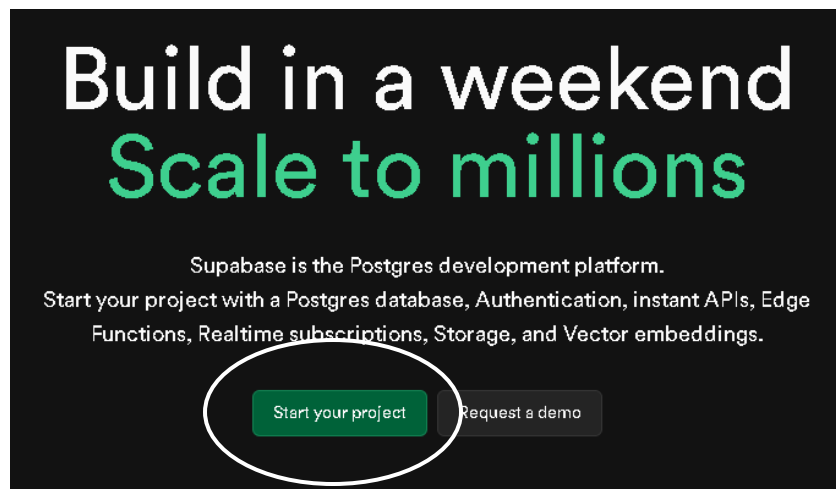
CRUD(クラッド)とは永続的なデータを取り扱うソフトウェアに要求される4つの基本機能である。
データの作成、読み出し、更新、削除の英語の頭文字をつなげた語
本課題では、Create、Readを行います。

CRUD操作

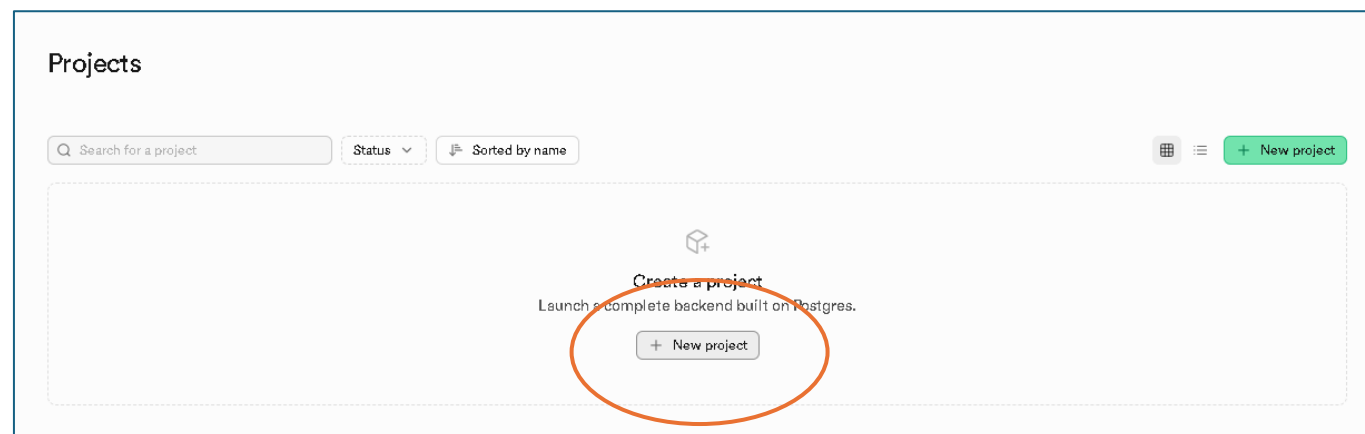
	Create	Read	Update	Delete
操作	データの作成	データの読み取り	データの更新	データの削除
SQL	INSERT文	SELECT文	UPDATE文	DELETE文

※CRUDにおけるCreateは、テーブルの作成とは異なる

Supabaseの始め方1



githubアカウント
と連携するなど
してサインイン



Supabaseの始め方2

Create a new project

Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization

Project name

Database password

This password is strong. [Generate a password.](#)

Region

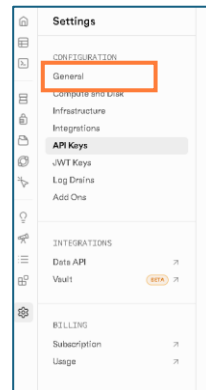
Select the region closest to your users for the best performance.

Security

- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schemas.
Recommended if using a client library like [supabase-js](#).
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

ProjectIDの取得



Project Settings

General configuration, domains, ownership, and lifecycle

General settings

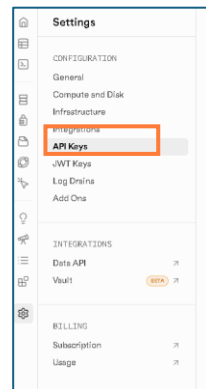
Project name
Displayed throughout the dashboard.

Project ID
Reference used in APIs and URLs.

Project ID

APIKey

APIKeyの取得 (anon public key)



API Keys

Configure API keys to securely control access to your project

Publishable and secret API keys **Legacy anon, service_role API keys**

anon public

This key is safe to use in a browser if you have enabled Row Level Security for your tables and configured policies. Prefer using [Publishable API keys](#) instead.

service_role **secret**

**** *
This key has the ability to bypass Row Level Security. Never share it publicly. If asked, generate a new JWT secret immediately. Prefer using [Secret API keys](#) instead.

Supabaseの始め方3

Vanilla JSでsupabaseClientをインスタンス化

1. public配下にsupabaseClient.jsの作成

```
const supabaseUrl = window.env.SUPABASE_URL;  
const supabaseKey = window.env.SUPABASE_ANON_KEY;  
const mySupabase = supabase.createClient(supabaseUrl, supabaseKey);
```

【TIPS】

環境変数を設定するファイル
(Reactなどで実装するなら
「.env」に書くことも多い)

2. 同一プロジェクトフォルダに「config.js」を作成

```
window.env = {  
  SUPABASE_URL: 'あなたのプロジェクトURL',  
  SUPABASE_ANON_KEY: 'あなたのAPI Key'  
};
```

URLは以下の形式
<https://<ProjectID>.supabase.co>

【例】Project ID: abcd1234なら
<https://abcd1234.supabase.co>

3. .gitignoreに「config.js」と追加

【TIPS】

GitHubにあなたの環境変数を公開するのが怖いのでgitで扱うのを無視(ignore)する

Supabaseの始め方4

Supabase内でデータベースを操作するには、SQL文(Postgre SQL)を書いて実行する。

データベースにデータを格納するには、適切なテーブルを予め作っておくことが必要

例

→この表のテーブルを作ってみる

id	店舗名
...	一号店

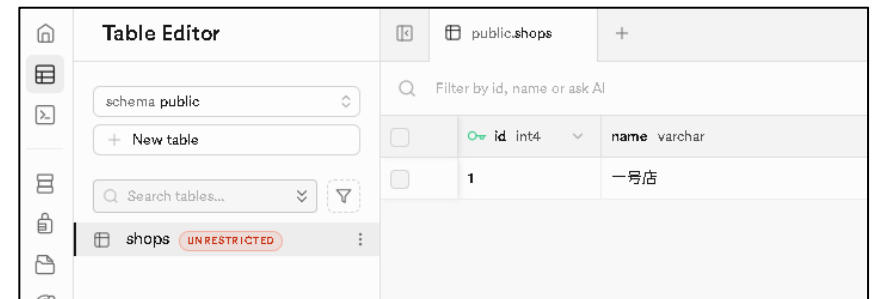
1. CREATE TABLEから作成するSQL文を書く

```
CREATE TABLE shops (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL  
);
```

2. サンプルデータを入れてみる

```
INSERT INTO shops (name) values('一号店');
```

3. Table editorからテーブルを確認してみる



Supabaseの始め方5

本課題は生のJavaScriptで
実装するので、参考程度に

Webアプリケーションとして、DBにデータをINSERTするリクエストを送る実装のコード例（Reactで実装した例）

例

→店舗名をshopsテーブルに格納

id	店舗名
...	一号店

店舗追加

追加

```
import React, { useState } from "react";
import { supabase } from "../supabaseClient";

export default function AddShop() {
  const [name, setName] = useState("");

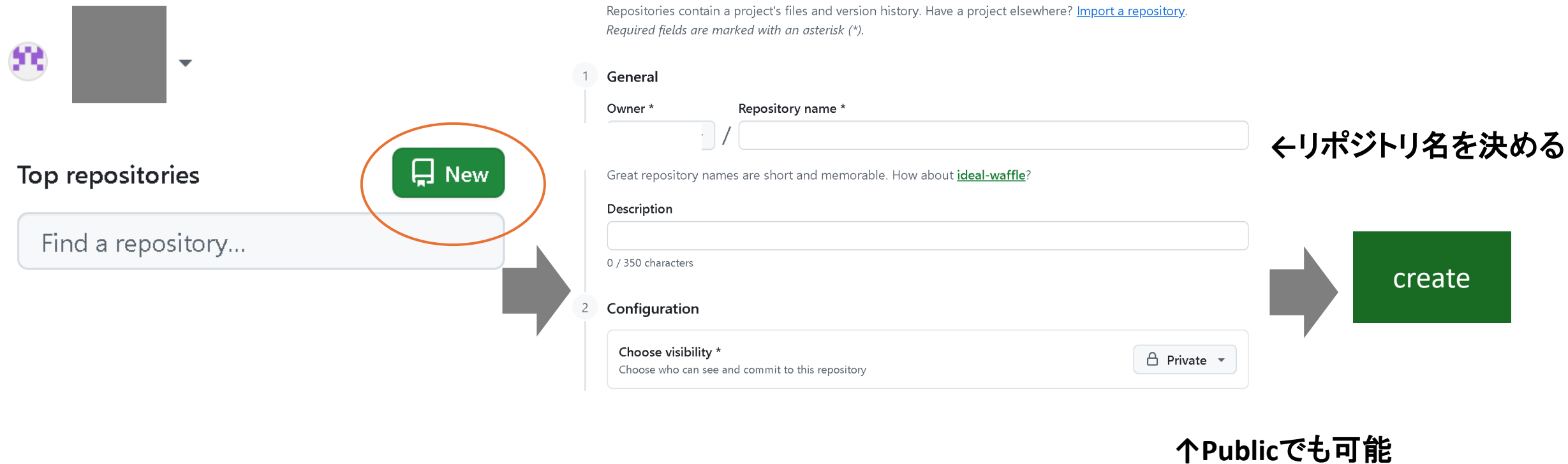
  const submit = async (e) => {
    e.preventDefault();
    if (!name) return;
    const { error } = await supabase.from("shops").insert([ { name } ]);
    if (error) return alert(error.message);
    setName("");
    alert("OK");
  };

  return (
    <form onSubmit={submit}>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button type="submit">追加</button>
    </form>
  );
}
```

Supabaseに接続するインスタンス

DB操作を行う箇所
その他のリクエストの送り方（例）
await supabase.from('shops').select('*')
(SELECT * FROM shopsと同じ要領)

Githubに新しくリモートリポジトリを立てる方法



新しく立てたりモトリポジトリにローカルの内容をPushする方法

Gitがインストールされ、初期設定を終え、ローカルのGitとGithubの連携・認証ができている前提。

Gitのインストール: <https://qiita.com/emiki/items/c0aa0b4da4b63921b153>

Git/GitHubの認証: <https://qiita.com/emiki/items/4103150a5d7a7cfbff24>

該当するディレクトリ(**Git配下におくディレクトリ**)で以下のコマンドを実行

```
git init
```

```
git add .
```

```
git commit -m “コミットメッセージ(なんでもいい)”
```

```
git branch -M main
```

```
git remote add origin https://github.com/あなたのユーザー名/リポジトリ名.git
```

```
git push -u origin main
```

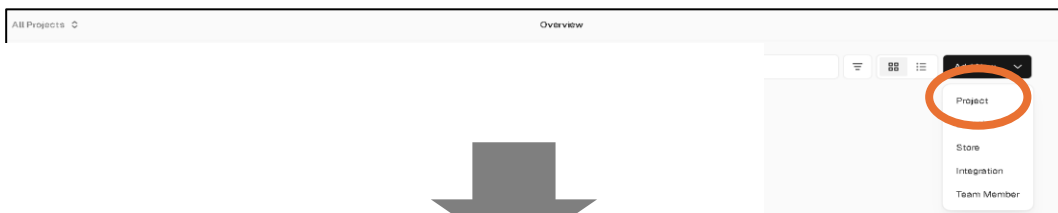
Vercelによるフロントエンドのデプロイ方法

Vercelのアカウント作成はGithubと連携すると楽。本説明はその後の話。

Root Directory

The directory within your project, where your code is located.

./



Framework Settings

When using a framework for a new project, it will be automatically detected. As a result, you can override them below.

Framework Preset

Other

ビルドとは、実行ファイルの生成
(コンパイルのようなもの)

Build Command ?

npm run vercel-build

Output Directory ?

public

Install Command ?

'yarn install', 'pnpm install', 'npm install', or 'bun install'

Development Command ?

None

←入力してください

←入力してください

Let's build something new

Enter a Git repository URL to deploy...

Import Git Repository

↑ここに張り付けるか
↓ここから選択

Search...

Import

Import

Import

Import

Import

Environment Variables

Key

Value

SUPABASE_URL

I9JU23NF394R6HH

SUPABASE_ANON_KEY

I9JU23NF394R6HH

+ Add More

Import .env

or paste the .env contents above. [Learn more](#)

←環境変数を設定してください

Deploy